# Curriculum Graph: Student-Centric Curriculum Planning with Dependency Analytics

**Mini Project for DS3294: Data Science Practice**

Ayush Raj Malhar Patel Sankar Maharajan Applen Tahu

3 April 2026

## 1 Introduction

We developed *Curriculum Graph*, a student-centric dashboard for major/minor planning that combines prerequisite dependency analytics with an interactive UI. The system helps students understand course dependencies, validate program requirements, and plan feasible semesters under offering and credit constraints.

Curriculum planning is often difficult for students because prerequisite relationships, semester offerings, and major/minor rules are spread across multiple documents. A graph-based representation is naturally suited for this problem: courses are modeled as nodes and prerequisites as directed edges. Graph traversal, topological logic, and bottleneck analysis then become useful for practical advising and semester-by-semester planning [1].

The main challenge is not only algorithmic; it is also usability. Even a correct graph can be hard to use if the interface is cluttered or if required insights are buried in technical detail. We therefore combine backend graph analytics with a student-facing dashboard built in Streamlit [3] and yFiles for interactive graph rendering [4].

## 2 Problem Statement and Objectives

Our goal is to build a reproducible, policy-aware planning system that answers the following student questions:

- What should I take next semester given my completed courses?
- Which prerequisites are blocking my progress?
- Is my current pathway valid for my major and minor?
- Can I still reach the degree target under term constraints?

The concrete objectives are:

- Build clean dependency graphs with semester and year structure.
- Validate major/minor pathways against rule sets.
- Generate constrained semester plans (course and credit caps).
- Provide an accessible UI with student mode and advanced mode.
- Ensure reliability through testing, validation checks, and a clean modular code structure.

# 3 Methods

## 3.1 Data

The system uses three primary JSON inputs:

- Course catalog: course code, name, credits, subject, kind, prerequisites, and semester offerings.
- Major/minor requirement rules: set-wise constraints and exclusions.
- College constraints: number of semesters and minimum degree credits.

In the current snapshot, the dataset contains 264 courses, 9 subjects, and 16 major/minor criteria entries [2]. Data-health checks report no duplicate course codes, no missing prerequisite links, and no unknown requirement codes.

## 3.2 Implementation Summary

The final codebase is organized into clear modules:

- `src/app.py`: Streamlit dashboard and interaction logic.
- `src/graph.py`: graph construction, planning, and analytics utilities.
- `src/validate_major_minor.py`: rule validation and requirement checks.

This structure enabled a maintainable workflow where UI improvements and algorithmic improvements could be developed independently and then integrated.

## 3.3 Data Quality and Rule Validation

Before graph construction and planning, we run a quality report over both the course catalog and the major/minor rule JSON. These checks are intended to catch data issues that would otherwise mislead students.

The quality report flags:

- duplicate course codes and duplicate program-rule entries,
- missing prerequisite links (a prerequisite code not present in the catalog),
- self-prerequisites (a course listing itself as a prerequisite),
- missing semester-offering tags,
- unknown requirement codes inside Set A–E rule lists.

The major/minor rules are encoded as a set-based schema:

- **Set D**: compulsory courses that must be included,
- **Set A**: elective pool with a minimum selection requirement,
- **Set B**: elective pool with a maximum allowed cap,
- **Set C**: supplementary pool that can contribute to totals,
- **Set E**: courses that are excluded (not counted) from totals.

In addition to set-wise constraints, the validator checks overall minimum course-count and credit requirements. The dashboard reports both the student input totals and the counted totals after applying Set E exclusions.

Requirement validation is performed set-wise (Sets A/B/C/D/E) and overall (minimum courses and minimum credits). Additionally, we support a prerequisite-completeness check that can be configured as *strict* (treated as errors), *advisory* (treated as warnings), or *off* (ignored). This prevents students from accidentally selecting advanced courses without ensuring all prerequisites are included in their plan.

## 3.4 Graph Modeling and Planning

We build a directed graph $G = (V, E)$ where $V$ is the selected course set and $E$ contains edges $(u, v)$ whenever course $u$ is a prerequisite of course $v$. The planning engine uses semester offerings and prerequisite satisfaction to select feasible courses term-by-term.

At a high level, the semester planner greedily selects available courses while respecting per-term caps and prioritizing compulsory requirements. The process is described in Algorithm 1.

For each course $c$, feasibility in semester $s$ is defined by two conditions: offering availability and prerequisite satisfaction,

$$c \in \mathcal{O}(s), \tag{1}$$

$$\mathrm{pred}(c) \subseteq \mathcal{C}_{\mathrm{done}}, \tag{2}$$

where $\mathcal{O}(s)$ is the set of courses offered in semester $s$, and $\mathcal{C}_{\mathrm{done}}$ is the completed course set.

### 3.4.1 Graph Integrity Checks (Cycle / Loop Detection)

Since prerequisite graphs must be acyclic to support strict scheduling, we explicitly detect directed cycles (loops) in the prerequisite graph. If any cycle is present, the dashboard highlights the cycle and the constrained planner exits early with a clear reason. When the graph is a DAG, we compute a topological order to provide a prerequisite-respecting sequence.

We also compute reachability from start-feasible root nodes (courses with no incoming prerequisites and at least one offering at/after the chosen start semester). Unreachable nodes indicate either offering constraints or a prerequisite chain that cannot be satisfied under the selected horizon.

Finally, we compute bottleneck indicators (downstream reach and betweenness centrality) to identify high-impact courses that unlock many others.

## 3.5 Dashboard Design

The dashboard has two usage modes:

- **Student mode**: fewer tabs, cleaner defaults, readable labels, and within-year arrow emphasis.
- **Advanced mode**: full algorithm diagnostics, rule explorer, validation internals, and quality views.

To reduce cognitive overload, graph legends are explicit, transition summaries are collapsible, and major/minor options are policy-filtered to approved offerings in the current deployment profile.

The UI uses progressive disclosure: essential views are visible by default, while heavier diagnostics are placed behind toggles/expanders. A key readability feature is the *Graph Clarity Controls*, which let users choose (i) a visual density profile, (ii) label detail level, (iii)

Table 1: Dataset and system summary statistics.

| Metric | Value |
|---|---|
| Total courses | 264 |
| Program entries (major/minor) | 16 |
| Subjects covered | 9 |
| Prerequisite edges | 94 |
| Average credits per course | 3.42 |
| Missing prerequisite links | 0 |
| Unknown requirement-code hits | 0 |

whether to show all prerequisite arrows or only within-year arrows, and (iv) layout style (hierarchic/orthogonal/organic/tree). This makes the same underlying graph usable for both quick planning and deeper debugging.

Interactive selection is central to the design: clicking nodes reveals course metadata (credits, offerings, subject/kind) and requirement-role labels (e.g., compulsory vs elective pool). This reduces the need to cross-reference external documents while planning.

### 3.6 Testing and Robustness

Reliability was treated as a first-class requirement because students interact with the system through an embedded graph renderer. The dashboard wraps graph rendering calls in a safe execution path that surfaces errors without breaking the entire page. On the analytics side, cycle detection prevents the planner from producing invalid schedules, and quality reports help identify catalog issues (missing prerequisites, duplicates, missing offerings) early. We also use lightweight smoke tests (syntax checks and headless app startup) to ensure the app remains runnable across machines.

## 4 Results

### 4.1 Quantitative Summary

Table 1 reports key metrics from the latest run.

### 4.2 Delivered System Features

The following deliverables were completed and validated in the current version:

- Student mode and advanced mode toggles for clean navigation.
- Policy-filtered major/minor options for institutional constraints.
- Year-semester structured prerequisite graphs with readable labels.
- Semester transition summaries for better planning visibility.
- Graph integrity checks: cycle (loop) detection, DAG/topological order diagnostics, and reachability summaries.
- Constrained term-by-term planner with blocked-course diagnostics.
- Bottleneck analytics (downstream reach and betweenness) to identify high-impact prerequisite courses.
- Major-minor combination simulator for comparative pathway analysis.

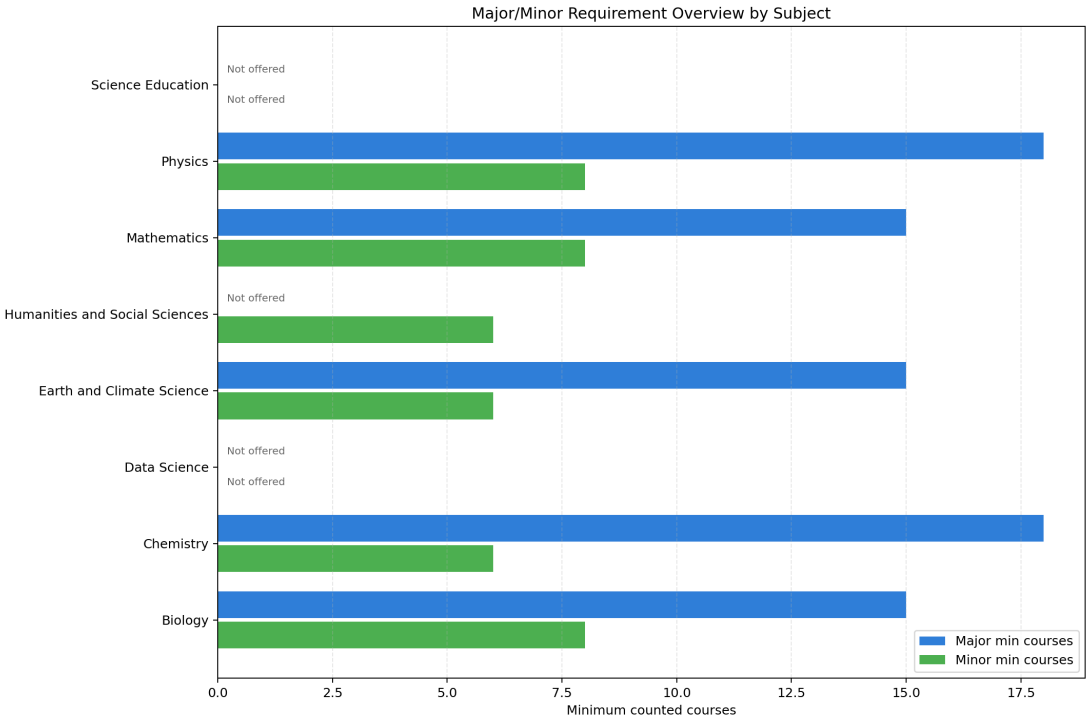


Figure 1: Overview of major-minor requirement structure used in the planning system.

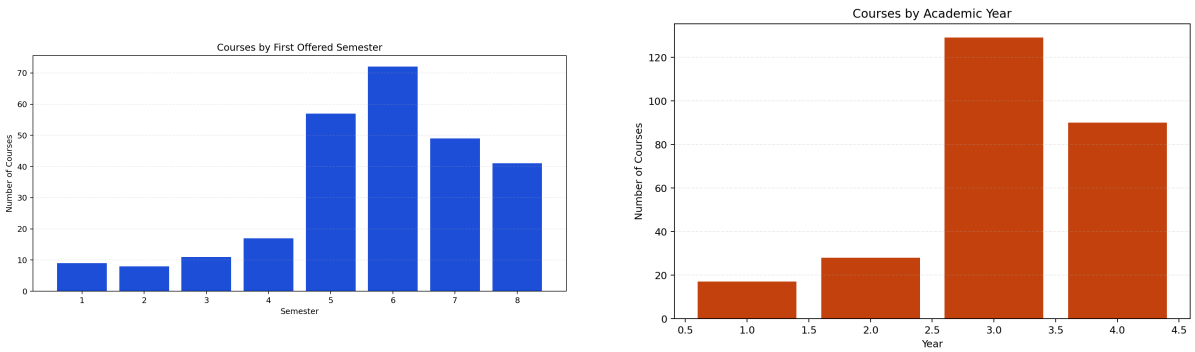


Figure 2: Course availability distribution over semester and academic year.

- Data quality checks and pathway validation reports.

## 4.3 Visual Analytics and Plots

Figure 2 shows the course distribution by semester and year, while Fig. 3 shows subject-level and prerequisite structure patterns.

## 4.4 Dashboard Outcome

The upgraded dashboard improves practical usability in three ways:

- Cleaner student-first navigation with optional advanced tabs.
- Better graph readability via consistent legends, role-aware colors, and transition summaries.

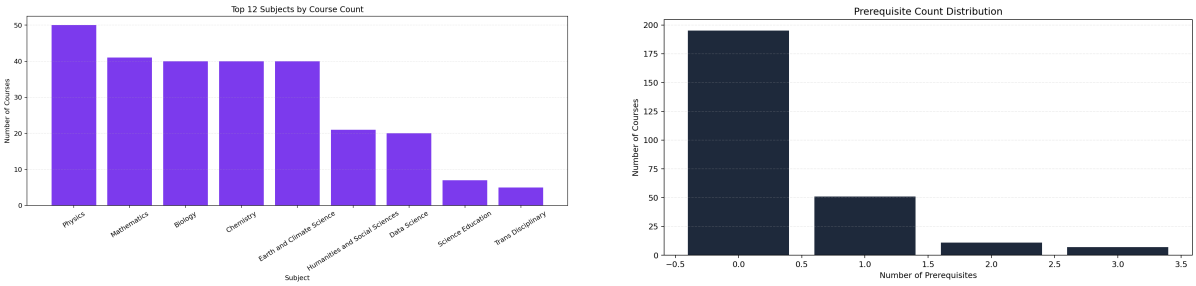


Figure 3: Subject concentration and prerequisite count structure.

- Cleaner module separation (UI vs. graph logic vs. validation) to support testing and maintainable iteration.

A typical student workflow is:

- start in student mode and pick a major/minor program,
- explore the roadmap graph clustered by year and semester,
- inspect transition summaries to understand which semesters depend on earlier courses,
- mark completed courses and generate a constrained term plan,
- review blocked-course explanations when constraints prevent full completion.

In student testing scenarios, the planner can surface blocked courses with unsatisfied prerequisites and propose semester-wise alternatives under credit and load constraints.

For readability, high-resolution dashboard screenshots are included in the Appendix as full-width figures (instead of compact side-by-side thumbnails).

### 4.5 Discussion of Outcomes

The final dashboard is not only visually cleaner but also operationally more reliable. Key reliability and usability outcomes include:

- No blocking data-quality issues in the current dataset snapshot.
- Stable headless startup and compile checks on the final codebase.
- Structured validation outputs that support testing and correctness.
- Better student readability through reduced clutter and clearer graph semantics.

### 4.6 Limitations

- The semester planner is a greedy, constraint-based heuristic; it produces feasible schedules when possible but does not guarantee a globally optimal plan.
- Outputs depend on the accuracy and completeness of the JSON snapshot (course offerings, prerequisite lists, and program rules). Changes in the catalog or handbook rules require updating these inputs.
- Some prerequisite checks are policy-driven (strict/advisory/off) and do not model co-requisites or timetable-level constraints (e.g., clashes).
- Very dense graphs can still be visually complex; the dashboard mitigates this via filters and clarity controls, but readability may vary with program size.

# 5 Conclusion and Outlook

Curriculum Graph Studio now functions as a complete planning and analytics stack: data quality checks, dependency graphs, constrained planning, pathway validation, and a student-friendly dashboard UI. The most significant contribution is the combination of graph-theoretic logic with a student-friendly interaction layer, which makes technical analytics actionable for academic decision-making.

Future work includes:

- personalized pathway recommendations based on student goals,
- what-if simulation for delayed/failed prerequisites,
- integration with institutional enrollment systems,
- longitudinal evaluation of advising outcomes.

# 6 Acknowledgements

We acknowledge the course and program-rule data sources provided for IISER Pune curriculum modeling, and the open-source tools that enabled this work.

# 7 Author Contributions

- Ayush Raj: UI development, testing, and codebase structuring.
- Malhar Patel: UI development, graph algorithms, and dependency analytics.
- Sankar Maharajan: Data preprocessing and graph validation.
- Applen Tahu: Data extraction and JSON schema design.

# 8 Code availability

Source code (GitHub): https://github.com/Mal-Pat/curriculum-graph/tree/main.

Key modules include `src/app.py` (dashboard UI), `src/graph.py` (graph construction and planning), and `src/validate_major_minor.py` (requirement validation).

# 9 How to Run (Streamlit Dashboard)

The dashboard is designed to be clone-safe and runnable locally with Python and Streamlit.

## 9.1 Clone and install

1. Clone the repository and enter it:

```
git clone https://github.com/Mal-Pat/curriculum-graph.git
cd curriculum-graph
```

2. Create and activate a virtual environment (macOS/Linux):

```
python -m venv .venv
source .venv/bin/activate
```

3. Install dependencies:

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

### 9.2 Run the app

```
python -m streamlit run src/app.py
```

Notes:

- Default dataset path: `data/IISER-P/`.
- Use *Student mode* for simplified navigation and graph defaults.
- Enable *Advanced analysis tabs* for cycle checks, topological ordering, reachability analysis, and detailed validation/data quality.

For a quick headless smoke test:

```
python -m streamlit run src/app.py --server.headless true --server.port 8522
```

## 10 References

## References

[1] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using networkx. *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008.

[2] IISER Pune. Iiser pune curriculum data files (course catalog, major/minor requirements, and constraints), 2026. Project dataset used in this report.

[3] Streamlit Inc. Streamlit documentation. https://docs.streamlit.io/, 2026. Accessed: 2026-04-03.

[4] yWorks GmbH. yfiles graphs for streamlit. https://github.com/yWorks/yfiles-for-html-demos/tree/master/demos/toolkit/streamlit, 2026. Accessed: 2026-04-03.

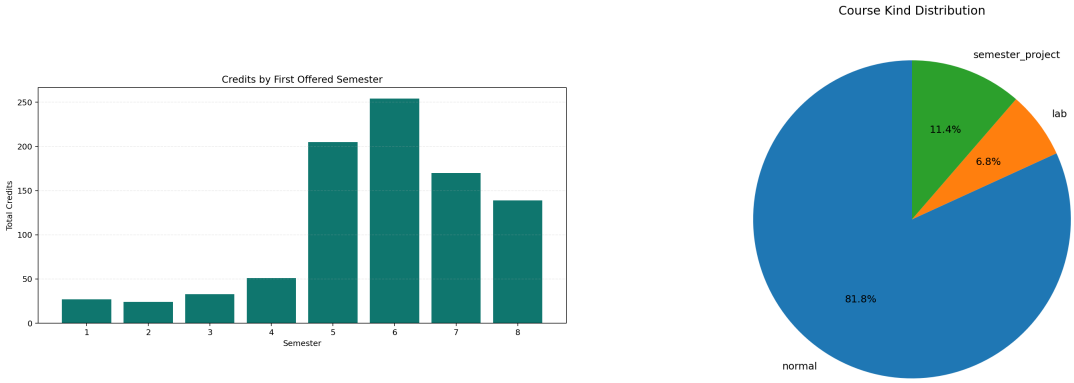


Figure 4: Credit load trend and course-kind distribution.

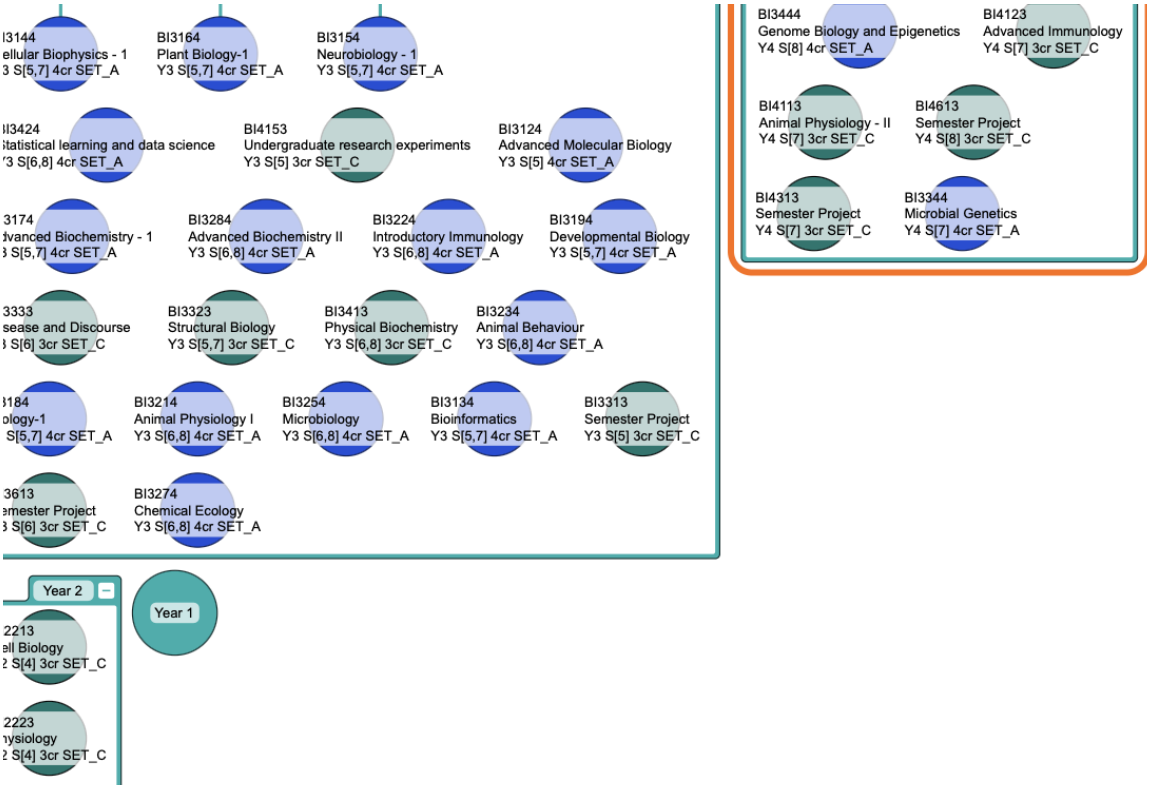


Figure 5: Year-wise prerequisite graph view showing set labels and reduced clutter (dashboard screenshot).

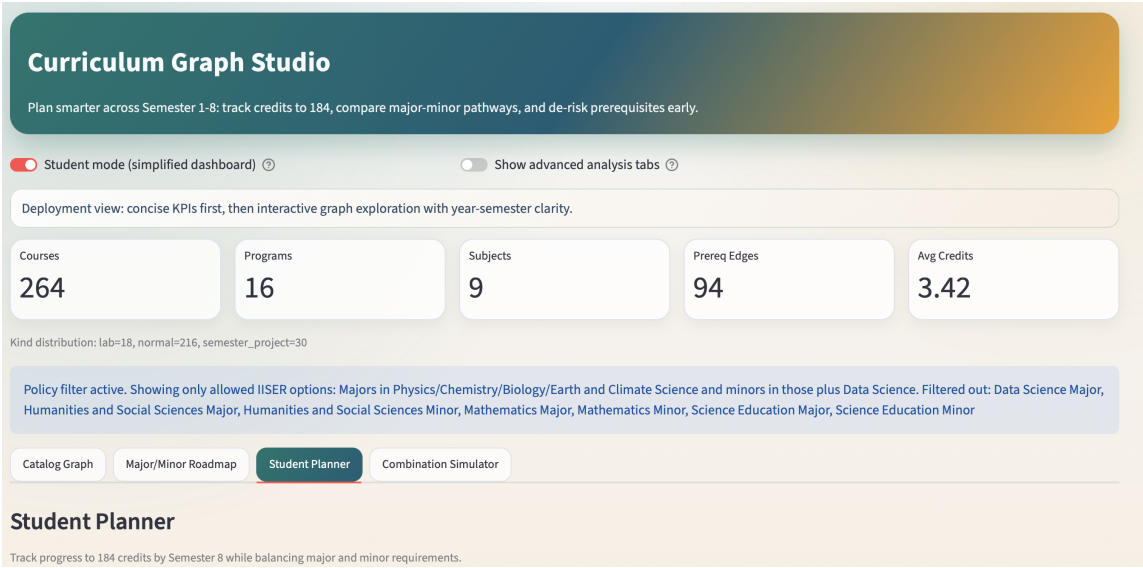


Figure 6: Student-mode dashboard overview showing KPIs, policy filtering, and the planner entry point.

# 11 Appendix

## 11.1 Additional figures

## 11.2 Algorithms

**Algorithm 1** Constrained Semester Planner

**Require:** Directed graph $G$, course index $I$, start semester $s_0$, term caps

**Ensure:** Semester plan $P$ and blocked set $B$

Initialize completed set $C$ from known history
**for** $s = s_0$ to planning horizon **do**
  Find feasible set $F_s = \{c \mid c \in \mathcal{O}(s) \wedge pred(c) \subseteq C\}$
  Rank $F_s$ by priority and downstream dependency impact
  Select courses under course-count and credit caps
  Add selected courses to $P[s]$ and update $C$
**end for**
$B \leftarrow V(G) \setminus C$
**return** $P, B$